

Week 8 課題 練習課題2

セキュリティテスト実施レポート

小見川 朋輝

2026年7月31日

1 テスト対象

本レポートは、Week 8 課題として作成した以下の2つのWebアプリケーションに対して実施したセキュリティテストの結果をまとめたものである。

対象アプリケーション

1. **会員制ブログ（基本課題）** — Laravel 12 + Laravel Breeze + Docker (nginx / PHP-FPM / MySQL)
ユーザー登録・ログイン機能付きブログ。ログインユーザーのみ投稿可能、自分の投稿のみ編集・削除可能。
2. **匿名掲示板（練習課題1）** — 素のPHP (PDO) + SQLite
もともと匿名で誰でも投稿できた掲示板に、ユーザー登録・ログイン機能を追加し、投稿者名を表示するように改修したもの。

2 テスト方法

実際にブラウザ・ターミナルから攻撃的な入力を送信し、アプリケーションの応答を確認するブラックボックステストを中心に、パスワードの保存形式についてはデータベースを直接参照する形で確認した。

3 テスト結果一覧

safeblue!15 項目	対象	手法	結果
SQLインジェクション	掲示板：ログインフォーム	ユーザー名に ' OR '1'='1'を入力してログインを試行	対策済み
XSS（クロスサイトスクリプティング）	掲示板：投稿フォーム	本文に<script>alert('XSS')</script>を投稿	対策済み
パスワードの保存方式	掲示板：users テーブル	DBを直接参照しpassword_hash 列の値を確認	対策済み

safeblue!15 項目	対象	手法	結果
CSRF対策	両アプリ：全フォーム	ソースコードレビューにより トークン検証の実装を確認	対策済み
認可バイパス (水平権限昇格)	ブログ：投稿編集・ 削除	別アカウントで他人の投稿の 編集URLに直接アクセス	対策済み

4 テスト詳細

4.1 1. SQLインジェクション（ログインバイパス）

手順と結果

匿名掲示板のログインフォームにおいて、ユーザー名欄に以下の文字列を入力しログインを試行した。

```
' OR '1'='1
```

もしSQL文が組み立て時に単純な文字列結合で構成されていた場合、
SELECT * FROM users WHERE username = '' OR '1'='1' のような
条件式に置き換わり、全ユーザーが条件にマッチしてログインが成立してしまう危険がある。
実際の結果としては、「ユーザー名またはパスワードが正しくありません。」と表示され、ログインは成立しなかった。

原因（実装の確認）

login.php 内で、SQL文の組み立てに PDOのプリペアドステートメント（**prepare / execute**）を使用しており、入力値がSQL文の構造そのものに影響を与えない設計になっている。

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');  
$stmt->execute([$username]);
```

これにより、入力値は常に「データ」として扱われ、SQL文の一部として解釈されない。

4.2 2. XSS（クロスサイトスクリプティング）

手順と結果

匿名掲示板の投稿フォームに、以下のスクリプトタグをそのまま投稿した。

```
<script>alert('XSS')</script>
```

もしエスケープ処理がされていなければ、この投稿を一覧ページで表示した際にJavaScriptが実行され、アラートダイアログが表示されてしまう。
実際には、アラートは表示されず、<script>alert('XSS')</script> という文字列がそのまま画面上のテキストとして表示された。

原因（実装の確認）

投稿内容を出力する際に **htmlspecialchars()** 関数を通しており、<、>、'などの特殊文字がHTMLエンティティに変換されている。

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
}
...
<?= e($post['body']) ?>
```

これにより、ブラウザは投稿内容を「実行可能なHTML/JavaScript」ではなく「表示するだけの文字列」として解釈する。

4.3 3. パスワードの保存方式**手順と結果**

掲示板のデータベース（SQLite）に対し、users テーブルを直接参照した。

```
$stmt = $pdo->query('SELECT username, password_hash FROM users');
```

結果、登録したパスワードは平文ではなく、以下のようなbcrypt形式のハッシュ値として保存されていることを確認した。

```
$2y$12$6hAtzkbB3xtdA3.zG6WLKuI1QBMWwCI6FZLgxK4B6lIhvNSEHuS3e
```

原因（実装の確認）

register.php でユーザー登録時に **password_hash()** 関数（デフォルトでbcryptアルゴリズム）を使用している。万が一データベースが漏洩しても、元のパスワードを直接読み取ることはできない。

4.4 4. CSRF対策**確認方法と結果**

CSRFは正常な画面操作からは再現しにくいため、ソースコードレビューによって実装を確認した。全てのPOSTフォーム（新規登録・ログイン・投稿）にワンタイムトークンが埋め込まれ、サーバー側で hash_equals() による検証が行われていることを確認した。

```
function verifyCsrfToken(?string $token): bool
{
    return isset($_SESSION['csrf_token']) && $token !== null
        && hash_equals($_SESSION['csrf_token'], $token);
}
```

トークンが一致しない、または存在しないリクエストは処理されない。

4.5 5. 認可バイパス（水平権限昇格）

手順と結果

会員制ブログにおいて、アカウントAで投稿を作成した後、別アカウントBでログインし、Aの投稿の編集URL（/posts/{id}/edit）に直接アクセスした。
結果、**403エラー**が返され、他人の投稿を編集・削除することはできなかった。

原因（実装の確認）

PostController の edit / update / destroy の各メソッドで、投稿の所有者（user_id）とログイン中のユーザーIDを比較するチェックを行っている。

```
abort_if($post->user_id !== auth()->id(), 403,  
        'この投稿を編集する権限がありません。');
```

5 総合評価と今後の改善点

総合評価

今回テストした5項目については、いずれも設計段階で意図した対策（プリペアドステートメント、出力エスケープ、パスワードハッシュ化、CSRFトークン、所有者チェック）が正しく機能していることを確認できた。

今後さらに強化できる点

- **ログイン試行回数制限（レート制限）**：現状は何度でもログインを試行できるため、パスワードの総当たり攻撃（ブルートフォース）への対策が未実装。Laravel側はthrottleミドルウェアの追加、掲示板側は失敗回数のカウントとロックアウト実装が有効。
- **HTTPS化**：現在は開発環境のためHTTP通信だが、本番運用時はTLS化によりログイン情報の盗聴を防ぐ必要がある。
- **パスワードポリシーの強化**：現状は8文字以上のみのチェック。文字種の組み合わせ要件などを追加するとより堅牢になる。
- **セッションタイムアウト**：一定時間操作がない場合に自動ログアウトする仕組みがあると、公共の端末利用時などのリスクを下げられる。